“In The Name Of God”

Assignment (2): Tiny Shakespeare Memory Duel

Professor: Dr.Mansoori

Student: Paniz Azarnia 40435097

---

1 Implementation:

1.1 preprocess.py:

a)
```
def load_text(path, data_fraction=0.1):
    with open(path, 'r', encoding='utf-8') as f:
        text = f.read()

    # Use only a fraction of dataset
    cut = int(len(text)*data_fraction)
    text = text[:cut]

    return text
```

Opens the file using UTF-8 encoding

Reads the full text

Keeps only a specified fraction (data_fraction) of the dataset to reduce training time

b)
```
def vocab_index(text):
    chars = sorted(list(set(text)))
    vocab_size = len(chars)

    ix_to_char={i:ch for i,ch in enumerate(chars)}
    char_to_ix={ch:i for i,ch in enumerate(chars)}

    return ix_to_char, char_to_ix, chars, vocab_size
```

Extracts all unique characters from the dataset

Sorts them to create a consistent vocabulary

Creates two dictionaries:

- char_to_ix: character → index
- ix_to_char: index → character

c)
```
def encode_text(text, char_to_ix):
    return np.array([char_to_ix[char] for char in text])
```

Replaces each character with its corresponding index using char_to_ix
Produces a numerical sequence representing the text

```
d) def create_sequence(encoded_text, seq_length = 5):
        X = []
        Y = []

        for i in range(len(encoded_text) - seq_length):
          x_seq = encoded_text[i:i+seq_length]
          y_seq = encoded_text[i+1:i+seq_length+1]

          X.append(x_seq)
          Y.append(y_seq)

        X = np.array(X)
        Y = np.array(Y)

        return X, Y
```

Splits the encoded text into fixed-length input sequences (X)
Generates corresponding target sequences (Y) shifted by one character
Each input sequence predicts the next character in the sequence

```
e) def preprocess(path,seq_length=5,data_fraction=0.5):
        text = load_text(path, data_fraction)
        ix_to_char, char_to_ix, chars, vocab_size = vocab_index(text)
        encoded_text = encode_text(text, char_to_ix)
        X,Y = create_sequence(encoded_text, seq_length)

        return X,Y,vocab_size,char_to_ix, ix_to_char
```

Runs the full preprocessing pipeline.
Steps:

- Loads the text dataset
- Builds the vocabulary mappings
- Encodes the text into numerical form
- Generates input–target training sequences

Returns:

- input sequences X
- target sequences Y
- vocabulary size
- character-to-index mapping
- index-to-character mapping

1.2 models.py:

```
b)class ElmanRNN(nn.Module):
        def __init__(self, vocab_size, hidden_size):
            super().__init__()

            self.hidden_size = hidden_size
            self.embedding = nn.Embedding(vocab_size, hidden_size)
            self.W_hh = nn.Linear(hidden_size, hidden_size)
            self.W_hx = nn.Linear(hidden_size, hidden_size)
            self.W_yh = nn.Linear(hidden_size, vocab_size)

        def forward(self,x,prev_hidden):
            seq_length, batch_size = x.size()
            outputs = []

            for t in range(seq_length):
                x_t = self.embedding(x[t])

                prev_hidden = torch.tanh(self.W_hx(x_t) + self.W_hh(prev_hidden))

                y_t = self.W_yh(prev_hidden)

                outputs.append(y_t)

            outputs = torch.stack(outputs)

            return outputs, prev_hidden


        def init_hidden(self, batch_size, device):
            return torch.zeros(batch_size, self.hidden_size, device=device)
```

Implements a simple **Elman Recurrent Neural Network** for **character-level** language modeling.

- Uses character embeddings as input representations
- Maintains a hidden state that is updated at each time step
- Generates output predictions based on the hidden state

The model consists of:

- Embedding layer for character encoding
- Hidden-to-hidden recurrent transformation (W_hh)
- Input-to-hidden transformation (W_hx)
- Hidden-to-output transformation (W_yh)

Forward (x, prev_hidden):

- Performs the forward pass of the Elman RNN.
- Steps:
  - Embeds the input characters
  - Updates the hidden state using the **previous hidden state** and current input with ***tanh activation function**
  - Computes output logits for each time step

- Stores outputs across the sequence
- Returns:
  - output logits for the entire sequence
  - final hidden state

init_output (self, batch_size, device)

- Initializes the hidden state with zeros.
- Returns a zero tensor of shape: **(batch_size, hidden_size)**

```
c)class JordanRNN(nn.Module):
  def __init__(self, vocab_size, hidden_size):
    super().__init__()

    self.hidden_size = hidden_size
    self.vocab_size = vocab_size
    self.embedding = nn.Embedding(vocab_size, hidden_size)
    self.W_hy = nn.Linear(vocab_size, hidden_size)
    self.W_hx = nn.Linear(hidden_size, hidden_size)
    self.W_yh = nn.Linear(hidden_size, vocab_size)

  def forward(self, x, prev_output):
    seq_length, batch_size = x.size()
    outputs = []

    for t in range(seq_length):
      x_t = self.embedding(x[t])

      hidden = torch.tanh(self.W_hx(x_t) + self.W_hy(prev_output))

      y_t = self.W_yh(hidden)

      prev_output = F.softmax(y_t, dim=-1)

      outputs.append(y_t)

    outputs = torch.stack(outputs)

    return outputs, prev_output

  def init_output(self, batch_size,device):
    return torch.zeros(batch_size, self.vocab_size, device=device)
```

Implements a simple **Jordan Recurrent Neural Network** for **character-level** language modeling.

- Uses the previous output instead of the previous hidden state for recurrence
- Maintains context through output feedback connections
- Predicts character probabilities sequentiallyThe model consists of:

The model includes:

- Embedding layer
- Output-to-hidden transformation (W_hy)
- Input-to-hidden transformation (W_hx)

- Hidden-to-output transformation (W_yh)

Forward (x, prev_output):

- Performs the forward pass of the Jordan RNN.
- Steps:
  - Embeds the current input characters
  - Computes the hidden state using the current input and previous output with ***tanh activation function**
  - Generates output logits
  - Applies softmax to create probability distributions for feedback
  - Stores outputs across the sequence
- Returns:
  - output logits for the entire sequence
  - final output probabilities

init_output (self, batch_size, device)

- Initializes the recurrent output context with zeros.
- Returns a zero tensor of shape: **(batch_size, vocab_size)**

* The **tanh activation function** is commonly used in recurrent neural networks because it introduces non-linearity while keeping the hidden state values bounded between −1 and 1. This helps stabilize training, prevents the hidden state from growing excessively, and allows the model to capture both positive and negative dependencies in sequential data. As a result, tanh is a standard activation function in classical RNN architectures such as Elman and Jordan networks.

1.3 train.py:

a)
```
class CharDataset(Dataset):
  def __init__(self, X,Y):
    self.X = torch.tensor(X, dtype=torch.long)
    self.Y = torch.tensor(Y, dtype=torch.long)
  def __len__(self):
    return self.X.shape[0]
  def __getitem__(self, idx):
    return self.X[idx], self.Y[idx]
```

Custom PyTorch dataset for character-level language modeling.

- Receives input sequences X and target sequences Y
- Converts them to PyTorch tensors
- Implements the required dataset methods:
  - __len__() returns the number of samples

- __getitem__() returns one (input, target) pair

This dataset is used with a DataLoader to create mini-batches for training.

```
b)def train_model(
    rnn_model="Elman",
    path="data/tiny-shakespeare.txt",
    seq_length=50,
    data_fraction=0.25,
    batch_size=64,
    hidden_size=256,
    num_epoch=10,
    lr = 0.0005,
    device=None
):
```

Main training function that prepares the data, builds the RNN model, trains it, evaluates it on validation data, and saves the trained model.

```
if device is None:
   device = "cuda" if torch.cuda.is_available() else "cpu"
```

Automatically selects the computation device:

- GPU (cuda) if available
- otherwise CPU

```
# 1) Load and Preprocess
X, Y, vocab_size, char_to_ix, ix_to_char = preprocess(path, seq_length, data_fraction)

dataset = CharDataset(X, Y)
train_size = int(len(dataset)*0.9)
val_size = len(dataset) - train_size
train_dataset, val_dataset = random_split(dataset, [train_size, val_size])

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
```

The dataset is prepared using the preprocessing pipeline:

- preprocess() loads the text
- builds the character vocabulary
- encodes the text into integer sequences
- generates training input (X) and targets (Y)

The dataset is then split into:

- 90% training data
- 10% validation data

Mini-batches are created using DataLoader, with the batch_size given as input to train_model.

```
# 2) Build Model
if rnn_model.lower() == "elman":
    model = ElmanRNN(vocab_size, hidden_size)
elif rnn_model.lower() == "jordan":
    model = JordanRNN(vocab_size, hidden_size)
else:
    raise ValueError("RNN type must be Elman or Jordan")

model.to(device)
```

The function supports two RNN architectures:

- Elman RNN
- Jordan RNN

The selected model is initialized using the vocabulary size and hidden dimension and then moved to the selected device.

```
# 3) Load and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=lr)

train_losses = []
val_losses = []

# perplexity
train_ppls = []
val_ppls = []

# Accuracy
train_accs = []
val_accs = []
```

Loss: CrossEntropyLoss for multi-class character prediction

Optimizer: Adam with a configurable learning rate

Lists are also initialized to store:

- training and validation loss
- perplexity
- character accuracy

```
# 4) Training loop
start_time = time.time()
for epoch in range(num_epoch):
    model.train()
    total_loss = 0.0

    train_total_correct = 0
    train_total_chars = 0
    for x_batch,y_batch in train_loader:
```

```
    x_batch = x_batch.to(device)
    y_batch = y_batch.to(device)


    #(batch_size, seq_length) -> (seq_length, batch_size)
    # That's how our RNN works
    x_batch = x_batch.transpose(0,1)
    y_batch = y_batch.transpose(0,1)

    # For last batch and mismatch error
    current_batch_size = x_batch.size(1)

    # because torch accumulates gradients
    optimizer.zero_grad()

    # assumption: batches are independent
    if rnn_model.lower() == "elman":
       prev_hidden = model.init_hidden(current_batch_size,device)
       outputs,_ = model(x_batch, prev_hidden)
    elif rnn_model.lower() == "jordan":
       prev_output = model.init_output(current_batch_size,device)
       outputs,_ = model(x_batch, prev_output)

    # reshape for loss:
    # outputs: (seq_len, batch_size, vocab_size)
    # -> (seq_len * batch_size, vocab_size)
    outputs = outputs.reshape(-1, vocab_size)
    # targets: (seq_len, batch_size) -> (seq_len * batch_size)
    y_batch = y_batch.reshape(-1)

    preds = torch.argmax(outputs, dim=1)

    train_total_correct += (preds == y_batch).sum().item()
    train_total_chars += y_batch.numel()


    loss = criterion(outputs, y_batch)
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 5)
    optimizer.step()

    # loss.item -> only need the numerical value of loss
    total_loss += loss.item()
```

The model is trained for a specified number of epochs, given as input (num_epoch).
For each batch:

1. Move data to the selected device
2. Transpose tensors to match RNN format(batch_size, seq_len) → (seq_len, batch_size)
3. Initialize the recurrent state
    - hidden state for Elman
    - previous output for Jordan
4. Run the forward pass

5. Reshape outputs for loss computation
6. Compute predictions and character accuracy
7. Compute loss
8. Perform backpropagation
9. Apply gradient clipping to prevent exploding gradients
10. Update model parameters

Training loss and accuracy are accumulated over batches.

```
# Validation loop
model.eval()

val_loss_total = 0
val_total_correct = 0
val_total_chars = 0

with torch.no_grad():

  for x_batch, y_batch in val_loader:

    x_batch = x_batch.transpose(0, 1).to(device)
    y_batch = y_batch.transpose(0, 1).to(device)

    current_batch_size = x_batch.size(1)

    if rnn_model.lower() == "elman":
      state = model.init_hidden(current_batch_size, device)
    else:
      state = model.init_output(current_batch_size, device)

    outputs, _ = model(x_batch, state)

    outputs = outputs.reshape(-1, vocab_size)
    targets = y_batch.reshape(-1)

    preds = torch.argmax(outputs, dim=1)


    val_total_correct += (preds == targets).sum().item()
    val_total_chars += targets.numel()

    loss = criterion(outputs, targets)

    val_loss_total += loss.item()
```

After each epoch, the model is evaluated on the validation dataset.

- Gradients are disabled using torch.no_grad()
- The same forward pass is performed
- Validation loss, perplexity, and accuracy are computed

This measures how well the model generalizes to unseen data.

```
# -----Train data-------
avg_train_loss = total_loss / len(train_loader)
# on unseen data
#train_ppl = torch.exp(torch.tensor(avg_loss))
train_ppl = math.exp(avg_train_loss)
train_losses.append(avg_train_loss)
train_ppls.append(train_ppl)

train_accuracy = train_total_correct/train_total_chars
train_accs.append(train_accuracy)

# -----Validation data------
avg_val_loss = val_loss_total / len(val_loader)
val_ppl = math.exp(avg_val_loss)

val_losses.append(avg_val_loss)
val_ppls.append(val_ppl)

val_accuracy = val_total_correct / val_total_chars
val_accs.append(val_accuracy)

# Compare vaidation and train
print(f"Epoch [{epoch + 1}/{num_epoch}]")
print(
   f"Train Loss: {avg_train_loss:.4f} | "
   f"Train PPL: {train_ppl:.4f} | "
   f"Train Acc: {train_accuracy:.4f}"
)

print(
   f"Val Loss: {avg_val_loss:.4f} | "
   f"Val PPL: {val_ppl:.4f} | "
   f"Val Acc: {val_accuracy:.4f}"
)
```

The following metrics are calculated:

- Loss – average cross-entropy loss
- Perplexity

$$PPL = e^{loss}$$

which measures how well the model predicts the next character.

- Character Accuracy (percentage of correctly predicted characters).

```
end_time = time.time()

total_time = end_time - start_time

print(f"Training Time: {total_time:.2f} seconds")
```

The total training time is measured using the time module to compare model efficiency.

```
# 5) Save model
torch.save({
   'model_state_dict': model.state_dict(),
   'vocab_size': vocab_size,
   'char_to_ix': char_to_ix,
   'ix_to_char': ix_to_char,
   'hidden_size': hidden_size,
   'rnn_model': rnn_model,
   'train_losses': train_losses,
   'train_ppls': train_ppls,
   'train_accuracies': train_accs,
   'val_losses': val_losses,
   'val_ppls': val_ppls,
   'val_accuracies': val_accs,
   'train_time': total_time
}, f'{rnn_model.lower()}_model.pth')

print(f"Model saved as {rnn_model.lower()}_model.pth")
```

After training, the model checkpoint is saved using torch.save().
The checkpoint includes:

- trained model weights
- vocabulary mappings
- model hyperparameters
- training and validation metrics
- total training time

This allows the model to be reloaded later for inference or text generation.

1.4 metrics.py:

```
def compare_elman_jordan(elman_path, jordan_path, device=None):
   if device is None:
      device = "cuda" if torch.cuda.is_available() else "cpu"

   elman = torch.load(elman_path, map_location=device)
   jordan = torch.load(jordan_path, map_location=device)


   elman_train_loss = elman["train_losses"]
   elman_train_ppls = elman["train_ppls"]
   elman_val_loss = elman["val_losses"]
   elman_val_ppls = elman["val_ppls"]
   elman_val_accuracy = elman["val_accuracies"]

   jordan_train_loss = jordan["train_losses"]
   jordan_train_ppls = jordan["train_ppls"]
   jordan_val_loss = jordan["val_losses"]
   jordan_val_ppls = jordan["val_ppls"]
```

```
jordan_val_accuracy = jordan["val_accuracies"]

# Elman Val vs Train loss
val_train_loss_plot(elman_train_loss, elman_val_loss, "Elman")
# Jordan Val vs Train loss
val_train_loss_plot(jordan_train_loss, jordan_val_loss, "Jordan")

# Elman Val vs Train perplexity
val_train_ppl_plot(elman_train_ppls, elman_val_ppls, "Elman")
# Jordan Val vs Train perplexity
val_train_ppl_plot(jordan_train_ppls, jordan_val_ppls, "Jordan")

# Compare Jordan vs Elman perplexity
compare_models(
    elman_train_ppls,
    jordan_train_ppls,
    "Train Perplexity Comparison",
    "Train Perplexity"
)

compare_models(
    elman_val_ppls,
    jordan_val_ppls,
    "Validation Perplexity Comparison",
    "Validation Perplexity"
)

compare_models(
    elman_val_accuracy,
    jordan_val_accuracy,
    "Accuracy Comparison",
    "Accuracy"
)
```

Main function that loads the trained Elman and Jordan RNN models and compares their training results using visualizations.

Steps performed:

1. Device Selection
   - Automatically selects GPU if available, otherwise CPU.
2. Load Trained Models
   - Loads saved checkpoints (.pth files) containing the training metrics for both models.
3. Extract Metrics
   - Retrieves stored values including:
     - training loss
     - validation loss
     - training perplexity
     - validation perplexity
     - validation accuracy

4. Plot Individual Model Performance
   - Train vs validation **loss curves**
   - Train vs validation **perplexity curves**
5. Compare Models
   - Generates comparison plots between Elman and Jordan RNN for:
     - training perplexity
     - validation perplexity
     - validation accuracy

```
def val_train_loss_plot(train_loss, val_loss, rnn_model):

  plt.figure(figsize=(8,5))

  plt.plot(train_loss, label="Train Loss")
  plt.plot(val_loss, label="Validation Loss")

  plt.xlabel("Epoch")
  plt.ylabel("Loss")

  plt.title(f"{rnn_model} Loss Curve")

  plt.legend()
  plt.grid(True)

  plt.show()
```

Plots the **training and validation loss curves** for a **single model.**

- Displays how the loss changes across epochs

```
def val_train_ppl_plot(train_ppl, val_ppl, rnn_model="Elman"):
  plt.figure(figsize=(8, 5))

  plt.plot(train_ppl, label="Train PPL")
  plt.plot(val_ppl, label="Validation PPL")

  plt.xlabel("Epoch")
  plt.ylabel("Perplexity")

  plt.title(f"{rnn_model} perplexity curve")
  plt.legend()
  plt.grid(True)

    plt.show()
```

Plots the **training and validation perplexity curves** for a **single model**.

- Perplexity measures how well the model predicts the next character in the sequence. Lower values indicate better performance.
- This plot shows how prediction quality improves during training.

```
def compare_models(elman_metric, jordan_metric, title, ylabel):

    plt.figure(figsize=(8,5))

    plt.plot(elman_metric, label="Elman")
    plt.plot(jordan_metric, label="Jordan")

    plt.xlabel("Epoch")
    plt.ylabel(ylabel)

    plt.title(title)

    plt.legend()
    plt.show()
```

Creates comparison plots between Elman and Jordan RNN for a given metric.
The function:

- plots both models on the same graph
- allows comparison of metrics such as perplexity or accuracy
- helps visually evaluate which model performs better.

```
if __name__ == "__main__":
    compare_elman_jordan("elman_model.pth", "jordan_model.pth")
```

When the script is run directly:

- It loads the saved model checkpoints
- Generates all comparison plots automatically

1.5 generate.py:

```
a) def generate_text(
        model_path,
        text_seed="to be",
        length=200,
        temperature=1.0,
        device=None):
```

Generates new text character-by-character using a trained Elman or Jordan RNN model.
The function loads a saved model checkpoint, reconstructs the network, and predicts the next character repeatedly to produce synthetic text.

```
if device is None:
    device = "cuda" if torch.cuda.is_available() else "cpu"
```

Automatically selects the computation device:

- GPU (cuda) if available
- otherwise CPU

```
# 1)Load Checkpoint
check_point = torch.load(model_path, map_location=device)

vocab_size=check_point['vocab_size']
char_to_ix = check_point['char_to_ix']
ix_to_char = check_point['ix_to_char']
hidden_size = check_point['hidden_size']
rnn_model = check_point['rnn_model']
```

The saved checkpoint file (.pth) is loaded using torch.load().

The checkpoint contains:

- vocabulary size
- character-to-index mapping (char_to_ix)
- index-to-character mapping (ix_to_char)
- hidden layer size
- model type (Elman or Jordan)

```
# 2)Rebuild model
if rnn_model.lower() == "elman":
    model = ElmanRNN(vocab_size,hidden_size)

else:
    model = JordanRNN(vocab_size,hidden_size)

model.load_state_dict(check_point['model_state_dict'])
model.to(device)
model.eval()
```

The correct RNN architecture is reconstructed based on the saved model type:

- ElmanRNN
- JordanRNN

The trained weights are restored using: **model.load_state_dict()**

The model is then switched to evaluation mode using: **model.eval()**

```
# 3)initial state
if rnn_model.lower() == "elman":
    state = model.init_hidden(batch_size=1, device=device)
else:
    state = model.init_output(batch_size=1, device=device)
```

The initial recurrent state is created before generation starts:

- Elman RNN initializes a hidden state
- Jordan RNN initializes a previous output state

The batch size is set to 1 because text is generated one character at a time.

```
# 4)convert seed text to indices
# error handling if char is unknown
seed_index = [char_to_ix[char] for char in text_seed if char in char_to_ix]
generated = text_seed
```

The input seed text is converted into numerical indices using the vocabulary mapping.
Example: "To be" → [45, 12, 5, ...]
Unknown characters are ignored to avoid indexing errors.
The seed text acts as the initial context for generation.

```
# 5)warm up with seed -> to start from a hidden state with context
for ix in seed_index[:-1]:
  # for test and prediction :  seq_length = 1 & batch_size = 1
  x = torch.tensor([[ix]], dtype=torch.long).to(device)
  outputs, state= model(x,state)

# We want to start prediction from the last character
if len(text_seed) != 0:
  current_index = seed_index[-1]
else:
  raise ValueError("text seed is empty")
```

Before generating new characters, the model processes the seed text character-by-character.
This updates the recurrent state so the model can learn the context of the initial text.
Only the last character of the seed is used to begin prediction.

```
# 6)generate character
for l in range(length):
  # for test and prediction :  seq_length = 1 & batch_size = 1
  x = torch.tensor([[current_index]], dtype=torch.long).to(device)
  outputs, state= model(x,state)

  # (1, 1, vocab_size) -> (vocab_size) : raw, unnormalized scores for each char
  logits = outputs.squeeze()

  probs = F.softmax(logits/temperature, dim=0)

  #next_ix = torch.multinomial(probs, 1).item()
  next_ix = torch.argmax(probs, dim=0).item()

  next_char = ix_to_char[next_ix]
  generated += next_char

  current_index = next_ix
```

```
return generated
```

For each generation step:

1. The current character index is fed into the model
2. The model predicts scores (logits) for all characters
3. Softmax converts logits into probabilities
4. The next character is selected
5. The predicted character is appended to the generated text
6. The predicted character becomes the next input

This process repeats until the desired text length is reached.

The implementation currently uses **torch.argmax()** to select the most probable character. This produces more stable and deterministic text.

An alternative approach is probabilistic sampling using **torch.multinomial()** which generates more diverse and creative outputs.

```
#8) Temperature Sampling

probs = F.softmax(logits/temperature, dim=0)
```

The temperature parameter controls randomness in generation:

- **temperature < 1.0**
  - more deterministic
  - safer and repetitive text
- **temperature > 1.0**
  - more random and creative
  - higher diversity but less coherence

Probability scaling:

$$P = \text{softmax}\left(\frac{logits}{temperature}\right)$$

```
if __name__ == "__main__":

    text = generate_text(
        model_path="elman_model.pth",
        text_seed="To be",
        length=300,
        temperature=0.8
    )
    print(text)
```

The function returns the complete generated text string.